Tutorial - 5

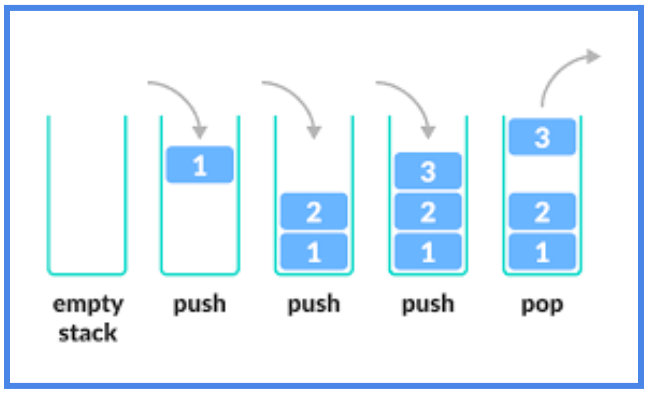
Navigating the Blocks of Data Structures: Stacks

Stack

A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle.
It can be visualized as a collection of items stacked on top of each other, like a stack of plates.

Operations:

- Push: Add an item to the top of the stack.
- Pop: Remove and return the item from the top of the stack.
- Peek: View the item at the top of the stack without removing it.



Common Use Cases:

- Function call management (call stack).
- Undo functionality in applications.
- Expression evaluation (e.g., postfix notation).

Implementation:
A stack can be implemented using various data structures and programming languages. Here are some common ways to implement a stack:

- Array Implementation:
    - You can use a one-dimensional array to implement a stack.
    - Use an index (usually called top) to keep track of the top element.
    - Push: Increment top and add the element to the array at the new top position.
    - Pop: Return the element at the top index and decrement top.
    - Pros: Simple and efficient for fixed-size stacks.
    - Cons: Limited by the size of the array.

- Linked List Implementation:
    - Use a singly linked list where each node contains the data and a pointer/reference to the next node.
    - The top of the stack is the head of the linked list.
    - Push: Add a new node to the head of the linked list.
    - Pop: Remove and return the head node.
    - Pros: Dynamic size, efficient for growing and shrinking stacks.
    - Cons: Slightly more memory overhead due to the pointers.

- You can also use a Doubly Linked List Implementation.
    - Similar to a singly linked list but each node has pointers to both the next and previous nodes.
    - Allows for efficient popping from both ends of the stack.
    - Useful for certain applications, but slightly more complex.

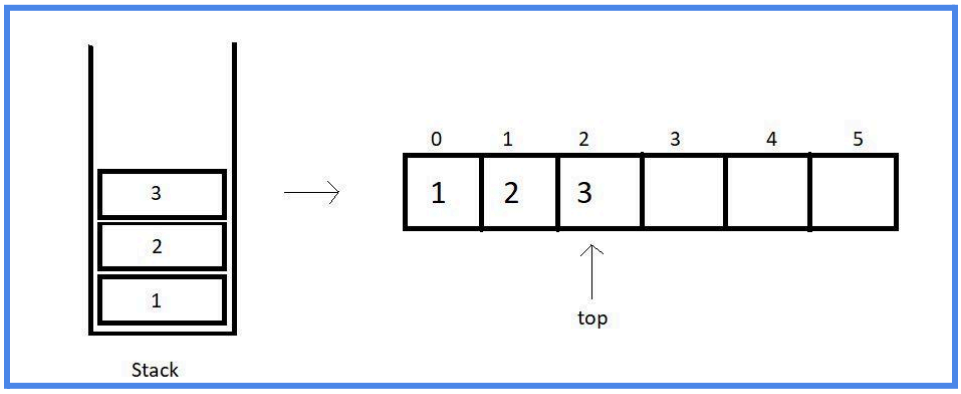
❖ Let's implement a Stack with an Array.

```c
#include <stdio.h>
#include <stdlib.h>

#define SIZE 6 // The number of elements in the array

int inp_array[SIZE]; //The inp_array will contain the elements of the stack.

int top = -1; // top will give us the location (index) of the last element inserted in the stack. As there is no element initially in the stack, we will initialize top to -1. It gets incremental each time an element is inserted in the array.
```



- Implementing the push function.

```
void push()
{
    int x;

    if (<Write your instruction here>) // Insertion is only possible if
there is some vacant space in the array.
    {
        printf("\nOverflow!!");
    }
    else
    {
        printf("\nEnter the element to be added onto the stack: ");
        scanf("%d", &x);

        <Write your instruction here> // increment the top pointer
        <Write your instruction here> // store the value that you want
at the top
    }
}
```

- Implementing the pop function.

```
void pop()
{
    if (<Write your instruction here>) //Deletion is possible when there
is at least one element in the array
    {
        printf("\nUnderflow!!");
    }
    else
    {
        <Write your instruction here> // print the element that is
removed
        <Write your instruction here> // decrement the pointer
    }
}
```

- Implement the peek function that prints the element at the top of the stack

```
void peek()
{
    if (<Write your instruction here>) //Base condition: No element found!
    {
        printf("\nNo element on the stack!");
    }
    else
    {
        printf("\nElement at the top of stack: \n");
        <Write your instruction here> //print the element at the top
     }
}
```

Additional Tasks:

- Implement a function display that prints all the elements of the stack.
- Implement a stack using a linked list. Refer to Tutorial-2 for implementing linked list.

Some Questions on Stack

Q1. Review the following straightforward code that aims to implement a stack using a structure. This code pushes a single element onto the stack and then proceeds to pop that element. However, there are minor syntax errors present in the code. Please identify and correct these issues to make the code operational.

```
#include<stdio.h>
struct stack
{
      int arr[100];
      int top;
};
int main()
{
      stack s;
      top = -1;
      // Let us push an element next
      top = top + 1
      s.arr[top]=10;
      // Let us pop and element next
      printf("Popped element = %d\n",s.arr[top]);
      top--;
```

```
      return(1);
}
```

Q2. Below is a code that implements stack initialization as a function call. What do you think is wrong with this code?

```
#include<stdio.h>

typedef struct stack
{
    int arr[100];
    int top;
}stack;

void initialise(stack x)
{
      x.top=-1;
}

int main()
{
    stack s; // Declare a variable of type 'stack'
    initialise(s); // Will this initialise the top of the stack? Do you see
an error?

    // Let us push an element next
    s.top = s.top + 1; // Increment the top
    s.arr[s.top] = 10; // Push 10 onto the stack

    // Let us pop an element next
    printf("Popped element = %d\n", s.arr[s.top]); // Print the popped
element
    s.top--; // Decrement the top to simulate a pop operation

    return 0; // Return 0 to indicate successful execution
}
```

To effectively tackle this question, it's essential to have a clear understanding of the concepts of "call by value" and "call by reference." Not sure what these terms mean?

You can get a grasp of them by reading the following link or just Google it:
https://www.geeksforgeeks.org/difference-between-call-by-value-and-call-by-reference/

Q3. Consider the following potential solutions to address this problem. Make an effort to grasp each solution and consult your instructor if you encounter any difficulties.

The problem with this code arises from the fact that the `initialise` function alters a duplicate of the `stack` structure, not the original one. Consequently, when you invoke `initialise(s)`, it doesn't truly initialize the `s` structure within the `main()` function; instead, it initializes a duplicate.

Way1 (Call by value): You can return the modified value of top and then collect it in the main() function to reassign to s.top.

Changes in the previous code

Before	After

void initialise(stack x)
{
x.top=-1;
}
	int initialise(stack x)
	{
	x.top=-1;
	return(x.top)
	}

initialise(s);	s.top = initialise(s);

Way 2 (Call by reference): Another way which avoids return statements is to use call by reference. Instead of passing s to the function, pass its address. Then whatever changes you make in s will be reflected

Before	After

void initialise(stack x)
{
x.top=-1;
}
	void initialise(stack *x)
	{
	(*x).top=-1;
	return((*x).top)
	}

initialise(s);	initialise(&s);

Please note that (*x).top can be alternatively written as x->top. x->top is a commonly used notation, so don’t let that arrow scare you.

Task: Modify the code given in Q2 and implement push and pop as functions.

Q4. Look at the following structure definition for a stack.

```
#include<stdio.h>
typedef struct stack
{
    int arr[100];
    int top;
}stack;
```

In the provided code, the maximum size of the stack is currently predefined as 100. How can you modify the code to allow the user to define the maximum size of the stack array at runtime? Basically the maxsize of the stack should be a user input. Try to find the solution by reading as few following hints as possible.

- Hint 1: Bring dynamic memory allocation to the picture.
  - stack s; s.arr=malloc…….
- Hint 2: What if we replace int arr[100] by int *arr which will allocate as much memory as you want at runtime?

Q5. You are tasked with implementing a stack data structure using a linked list in C. Below is a partial code for the stack structure. Some essential lines are missing, and you need to complete the implementation.

```
#include <stdio.h>
#include <stdlib.h>

// Define a structure for a stack node
struct Node {
    int data;
    struct Node* next;
};

// Define the stack structure
struct Stack {
    struct Node* top;
};
```

```
// Function to create an empty stack
struct Stack* createStack() {
    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
    if (!stack) {
```

```
        // Handle memory allocation failure
    }
    stack->top = NULL;
    return stack;
}

// Function to push an element onto the stack
void push(struct Stack* stack, int item) {
    // Complete the implementation for pushing an element onto the stack
}

// Function to pop an element from the stack
int pop(struct Stack* stack) {
    // Complete the implementation for popping an element from the stack
}

int main() {
    struct Stack* stack = createStack();

    // Test the stack operations here

    return 0;
}
```

Fill in the missing lines in the `push` and `pop` functions to complete the implementation of the stack using a linked list. Ensure that pushing and popping elements from the stack work correctly and handle edge cases appropriately. Once you've filled in the missing lines, test the stack operations in the `main` function to verify that the stack behaves as expected.

Q6. You are tasked with implementing a program in C to convert an infix mathematical expression to a postfix expression using the stack data structure. Below is a partial code for the conversion process. Some essential lines are missing, and you need to complete the implementation.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>

// Define a structure for a stack node
struct Node {
    char data;
```

```
    struct Node* next;
};

// Define the stack structure
struct Stack {
    struct Node* top;
};
```

```
// Function to create an empty stack
struct Stack* createStack() {
    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
    if (!stack) {
        // Handle memory allocation failure
    }
    stack->top = NULL;
    return stack;
}
```

```
// Function to push an element onto the stack
void push(struct Stack* stack, char item) {
    // Complete the implementation for pushing an element onto the stack
}

// Function to pop an element from the stack
char pop(struct Stack* stack) {
    // Complete the implementation for popping an element from the stack
}

// Function to check if a character is an operator (+, -, *, /, etc.)
bool isOperator(char c) {
    // Complete the implementation for checking if a character is an
operator
}

// Function to convert an infix expression to a postfix expression
void infixToPostfix(char* infix, char* postfix) {
    // Complete the implementation for the infix to postfix conversion
}
```

```
int main() {
```

```
    char infix[100];
    char postfix[100];

    printf("Enter an infix expression: ");
    gets(infix);

    infixToPostfix(infix, postfix);

    printf("Postfix expression: %s\n", postfix);

    return 0;
}
```

Your task is to fill in the missing lines in the `push`, `pop`, `isOperator`, and `infixToPostfix` functions to complete the implementation of the infix to postfix conversion. Ensure that the conversion correctly handles operators and parentheses and follows the rules of operator precedence.

Once you've filled in the missing lines, test the program by entering different infix expressions to verify that the conversion to postfix is performed correctly.

Takehome Problems

Q7. Here are three example code scenarios to implement a stack

Scenario 1: Fixed-Size Stack with a Simple Variable (`arr[100]`, `stack s`)

```
#include <stdio.h>

typedef struct stack {
    int arr[100];
    int top;
} stack;

int main() {
    stack s;
    s.top = -1;

    // Implement stack operations here...

    return 0;
}
```

Scenario 2: Fixed-Size Stack with a Dynamically Allocated Array (`*arr`, `stack s`)

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct stack {
    int* arr;
    int top;
} stack;

int main() {
    stack s;
    s.top = -1;
    s.arr = (int*)malloc(sizeof(int) * 100);

    if (s.arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    // Implement stack operations here...

    free(s.arr); // Don't forget to free the allocated memory

    return 0;
}
```

Scenario 3: Dynamically Sized Stack and Array (`*arr`, `stack *s`)

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct stack {
    int* arr;
    int top;
} stack;

int main() {
    stack* s = (stack*)malloc(sizeof(stack));
    s->top = -1;
```

```
    s->arr = (int*)malloc(sizeof(int) * 100);

    if (s->arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    // Implement stack operations here...

    free(s->arr); // Don't forget to free the allocated memory
    free(s);

    return 0;
}
```

Understand the key differences among these scenarios in terms of memory allocation, flexibility, and potential use cases. Which scenario would you choose for a small, fixed-size stack, and which one for a stack with dynamic size requirements? Try to implement a stack using all these approaches and understand the differences in the syntax.

Q8. Sort a Stack [Tricky but Important; *Asked in many interviews*]
You are given a stack of integers, and your task is to sort it in ascending order using only stack operations (push, pop, peek, and is_empty). You are not allowed to use any other data structure (e.g., arrays, lists) for sorting.

Note: You may consider using an auxiliary stack to help with sorting while maintaining the original stack's elements.
